

Project Structure

Learn about the Storefront Next project structure to navigate code quickly and follow conventions that keep your codebase maintainable as it grows.

Overview

A Storefront Next project is built on [React Router 7](#) in framework mode, and uses [Vite](#) as the build tool. The structure separates your app code (`src/`) from configuration files at the project root.

Contents

Overview

- Application Code in `src/`
- File Naming Conventions
- Root File (`root.tsx`)
- Configuration Files
- Where to Put Code Elements
- Example: Adding a Wishlist Feature
- Customizing Entry Files
- See Also

```
1 my-storefront/
2   └─ src/
3     └─ routes/
4       └─ components/
5         └─ lib/
6           └─ hooks/
7             └─ providers/
8               └─ middlewares/
9                 └─ locales/
10                  └─ root.tsx
11                   └─ routes.ts
12                    └─ app.css
13   └─ public/
14   └─ types/
15   └─ config.server.ts
16   └─ vite.config.ts
17   └─ react-router.config.ts
18   └─ package.json
```

Application code
Page components
Reusable UI components
Utilities and helpers
Custom React hooks
React context providers
Request/response interceptors
Translation files
Application root file
Route configuration
Global styles
Static assets
TypeScript type definitions
Commerce API configuration
Build configuration
React Router configuration
Dependencies

Application Code in `src/`

Routes

The `routes/` directory contains your page components. Storefront Next uses React Router's [file-based routing](#), where file names determine URL paths.

```
1 src/routes/
2   └─ _index.tsx
3   └─ category.$categoryName.tsx
4   └─ product.$productId.tsx
5   └─ cart.tsx
6   └─ checkout.tsx
7   └─ account.tsx
```

Home page (/)
Category page (/category/\$categoryName)
Product page (/product/\$productId)
Cart page (/cart)
Checkout (/checkout)
Account layout (/account)



File Naming Conventions

Pattern	Example	URI
<code>_index.tsx</code>	<code>_index.tsx</code>	<code>/</code>
<code>name.tsx</code>	<code>cart.tsx</code>	<code>/c</code>
<code>\$param</code>	<code>product.\$productId.tsx</code>	<code>/p</code>
<code>\$.tsx</code>	<code>category.\$.tsx</code>	<code>/c</code>
<code>parent.child.tsx</code>	<code>account.orders.tsx</code>	<code>/a</code>
<code>action.*</code>	<code>action.cart-item-add.tsx</code>	<code>-</code>

Note

Files starting with underscore (`_`) have special meaning. `_index.tsx` renders at the parent path, while a `_layout.tsx` file creates a layout wrapper without adding a URL segment.

The `routes.ts` file configures routing behavior. The `flatRoutes()` function scans your `src/routes/` directory and generates route configuration automatically based on file names. You rarely have to modify this file.

```
1 // src/routes.ts
2 import { type RouteConfig } from '@react-router'
3 import { flatRoutes } from '@react-router/fs-routes'
4
5 export default flatRoutes() satisfies RouteConfig
```

Components

The `components/` directory contains reusable UI elements organized by feature area.

```
</>
```



```
6 |   └─ product-tile/          # Product card:
7 |   └─ cart/                 # Cart component
8 |   └─ ...
```

The `ui/` subdirectory contains [shadcn/ui](#) components –accessible, customizable primitives that you can modify directly.

Create a component when:

- The UI element is used in multiple places.
- The component encapsulates complex logic or state.
- You want to test the UI in isolation (via Storybook).

Lib

The `lib/` directory contains utilities, API clients, and business logic that isn't tied to React.

```
</> 1 src/lib/
      2 └─ api/          # Commerce API
      3 └─ utils/        # General util:
      4 └─ ...
```

Place code here when it:

- Doesn't use React hooks or components.
- Could be shared across multiple routes or components.
- Handles data transformation or business logic.

Hooks

Custom React hooks extract reusable stateful logic from components.

```
</> 1 src/hooks/
      2 └─ use-product.ts    # Product data
      3 └─ use-cart.ts      # Cart operati
      4 └─ use-navigation.ts # Navigation u
      5 └─ ...
```

When to create a custom hook:

- Multiple components need the same stateful logic.
- You want to encapsulate complex operations (like API calls with loading states).
- You must share behavior across routes.

Providers



```
1 src/providers/
2   └─ cart-provider.tsx      # Cart state and actions
3   └─ locale-provider.tsx    # Internationalization
4   └─ ...
```

Providers wrap the app in `root.tsx` and make data available to any nested component without prop drilling.

Middlewares

Request and response middleware that runs before route loaders:

```
1 src/middlewares/
2   └─ auth-middleware.server.ts # Authentication
3   └─ auth-middleware.client.ts # Authentication
4   └─ i18n-middleware.ts        # Locale detection
5   └─ ...
```

Middleware is registered in `root.tsx` and executes in order for each request. Use middleware to:

- Parse authentication tokens
- Detect user locale
- Set up shared context that loaders need

For details on implementing middleware, see [Storage and Sessions](#).

Locales

Translation files for internationalization:

```
1 src/locales/
2   └─ en-US.json
3   └─ fr-FR.json
4   └─ ...
```

See [Internationalization](#) for setup and usage.

Root File (root.tsx)

The `root.tsx` file is your app's entry point. It exports several key pieces that define your app's behavior:

Export	Purpose
<code>middleware</code>	Server middleware chain (auth, locale detection)

[Try for free](#)

<code>loader</code>	Root data loader (fetches data needed by all pages)
<code>Layout</code>	HTML document structure (<code><html></code> , <code><head></code> , <code><body></code>)
<code>default (App)</code>	Application shell with header, footer, and <code><Outlet /></code> for page content

Most customization happens in middleware and the App component. Typically, you only need to modify Layout when changing document-level meta tags or adding scripts.

Configuration Files

File	Purpose
<code>config.server.ts</code>	Commerce API connection and storefront settings. See Configuration .
<code>vite.config.ts</code>	Build plugins, path aliases, Server-Side Rendering (SSR) settings. See Build Tools .
react-router.config.ts 	React Router framework options
<code>tsconfig.json</code>	TypeScript compiler options
<code>package.json</code>	Dependencies and npm scripts

Where to Put Code Elements

You want to...	Put it in...
Add a new page	<code>src/routes/</code> (file name = URL path)
Create a reusable component	<code>src/components/</code>
Add a server action	<code>src/routes/action/*.tsx</code>
Add a utility function	<code>src/lib/</code>

[Try for free](#)

Add translations	<code>src/locales/</code>
Add static assets (images)	<code>public/</code>
Add global styles	<code>src/app.css</code>
Configure commerce or storefront settings	<code>config.server.ts</code>

Example: Adding a Wishlist Feature

Here's how the project structure guides you when adding a new feature:

```
1 src/
2 |─ routes/
3 |   │─ wishlist.tsx           # 1. Page at
4 |   └─ action.wishlist-add.tsx # 2. Server a
5 |─ components/
6 |   └─ wishlist/
7 |       │─ wishlist-item.tsx  # 3. UI compo
8 |       └─ wishlist-empty.tsx
9 |─ hooks/
10 |   └─ use-wishlist.ts        # 4. Hook fo
11 |─ lib/
12 |   └─ api/
13 |       └─ wishlist.ts        # 5. API cli
```

This separation keeps your code organized.

- Routes define what URLs exist and load data.
- Components handle presentation.
- Hooks manage stateful logic.
- Lib contains pure functions and API calls.

Customizing Entry Files

React Router provides default entry files for client hydration and server rendering. These entry files are hidden by default but can be revealed for advanced customization.

```
1 npx react-router reveal
```

This command creates `entry.client.tsx` and `entry.server.tsx` in your `src/` directory.

When to customize:

[Try for free](#)

For most projects, the default entry files work without modification.

See Also

- [React Router API Reference: React Router File Route Conventions](#)
- [React Router API Reference: React Router root.tsx](#)
- [_Vite:_Vite Configuration](#)

DID THIS ARTICLE SOLVE YOUR ISSUE?

Let us know so we can improve!

[Share your feedback](#)

DEVELOPER CENTERS

Heroku
MuleSoft
Tableau
Commerce Cloud
Lightning Design System
Einstein
Quip

POPULAR RESOURCES

Documentation
Component Library
APIs
Trailhead
Sample Apps
AppExchange

COMMUNITY

Trailblazer Community
Events and Calendar
Partner Community
Blog
Salesforce Admins
Salesforce Architects

© Copyright 2026 Salesforce, Inc. [All rights reserved](#). Various trademarks held by their respective owners. Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#) [Contact](#) [Use of Cookies](#)

[Cookie Preferences](#) [Your Privacy Choices](#)

